Algoritmos

2026-04-01

Introducción

Este es un documento en R Markdown. Permite generar HTML o Word combinando texto y código.

Primeros comandos de R

R es un leguaje muy parecido a matla y trabaja con variables que en general pueden ser matrices.

```
A <- 38
A <- 40
B <- 60
C <- B - A
C
```

```
## [1] 20
```

```
summary(cars)
```

```
##      speed           dist       
##  Min.   : 4.0   Min.   :  2.00  
##  1st Qu.:12.0   1st Qu.: 26.00  
##  Median :15.0   Median : 36.00  
##  Mean   :15.4   Mean   : 42.98  
##  3rd Qu.:19.0   3rd Qu.: 56.00  
##  Max.   :25.0   Max.   :120.00  
```

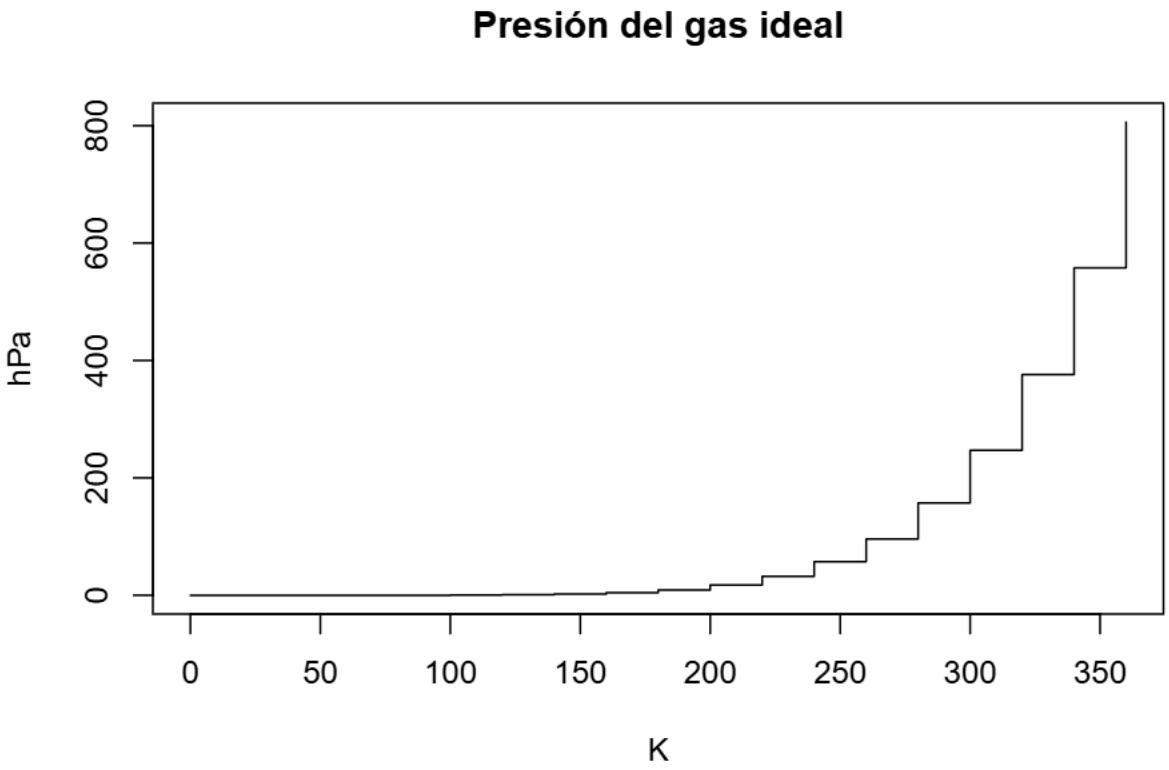
```
Y <- 40
alpha <- 30
```

Uso de data sets o base de datos internos de R

Usando el comando `data()` en la consola obtenemos distintos datos ya cargados en la base de datos de R, estos datos estan cargados en tablas de las cuales podemos elegir columnas especificas si escribimos el signo `$` despues del comando de informacion especifico que queremos

Gráficos

```
plot(pressure, type = "s",
     main = "Presión del gas ideal",
     ylab = "hPa",
     xlab = "K")
```



Comando plot

Es un comando que sirve para graficar cualquier fuente de datos que le asignemos: Tenemos la posibilidad de asignar nombres a las variables o hacer cambios del grafico, en caso de no saber el comando podemos escribir **?plot** en la consola, a la derecha nos va a abrir una pestaña y ahí tocar Generic X-Y Plotting y ahi nos va a explicar cada comando

Funciones estadísticas

```
z1 <- rnorm(350, 22, 5)
z1
```

```
##   [1] 23.950588 31.176043 20.178728 23.534439 13.341041 18.850508 24.166953
##   [8] 14.449217 17.802308 20.599770 27.127563 23.725750 24.660428 25.765256
##  [15] 15.904660 40.966963 19.262532 21.371298 22.288554 28.319467 31.278900
##  [22] 27.962291 18.100298 26.010156 15.698531 22.265542 22.859857 23.879975
##  [29] 27.138497 20.498410 20.882818 17.967835 26.591382 20.035137 22.394608
##  [36] 19.166907 12.993995 29.460240 27.376663 27.670742 32.797760 16.869348
##  [43] 21.069878 13.871969 34.958414 28.856921 14.226226 18.138891 17.992551
##  [50] 23.355206 19.066860 21.339256 18.112417 24.323866 17.848671 19.875924
##  [57] 24.769507 21.092537 18.783474 14.666499 26.482501 15.710247 25.435195
##  [64] 23.682828 33.003363 23.144814 35.381476 24.082670 17.136828 25.109546
##  [71] 16.275779 20.668767 24.882828 17.998946 25.873253 19.869467 13.559929
##  [78] 21.741907 24.572942 27.064343 22.112555 16.548136 25.012835 29.000658
##  [85] 14.418745 32.042765 20.054495 23.798389 26.337460 20.599354 26.989339
##  [92] 13.790477 22.920953 22.066694 17.943803 25.710856 24.689163 28.931191
##  [99] 18.903074 18.695884 17.668154 24.527607 18.931475 22.337421 12.483957
## [106] 16.963504 28.113427 25.126777 21.415514 23.163284 22.240171 22.937890
## [113] 20.684421 20.583978 27.210373 18.004936 21.309370 14.542333 12.478507
```

```
## [120] 30.271916 19.540959 16.004865 28.561485 22.513236 26.396857 28.261497
## [127] 20.113664 23.632785 20.966368 25.446376 23.209743 33.398354 21.747364
## [134] 24.948667 15.833494 29.310492 20.406936 21.458018 17.544193 19.475598
## [141] 27.777402 24.268895 23.174153 21.356881 22.092547 17.157153  8.457479
## [148] 25.078754 19.030680 14.413572 20.582167 27.274940 22.998059 28.207154
## [155] 16.646228 26.396595 15.325222 21.918037 22.578677 21.310977 24.380334
## [162] 28.163804 23.911849 18.881784 20.547581 32.252161 24.707356 23.260217
## [169] 23.903145 24.563081 25.736890 19.012532 31.218350 16.194517 17.161029
## [176] 18.578661 21.643285 26.443019 16.200119 18.349940 11.613089 28.495806
## [183] 19.169087 23.348524 10.809877 11.728924 20.818348 16.628141 13.790797
## [190] 16.647019 19.213118 16.602250 23.683495 31.691953 30.136824 25.357040
## [197] 22.259587 24.477142 21.123375 27.929481 25.381480 18.980475 21.899198
## [204] 15.185956 20.511874 19.502381 27.464912 22.547381 22.318537 23.892753
## [211] 20.414838 34.379671 29.735793 22.455326 21.937532 22.593634 17.519431
## [218] 30.486530 22.998123 15.211598 22.725015 17.476312 18.575291 25.320595
## [225] 22.466488 19.879220 24.545668 20.536920 22.038101 18.177967  9.657494
## [232] 15.088975 29.014301 12.630794 16.956540 27.445244 13.630765 15.431528
## [239] 21.943489 16.032188 26.900617 13.615713 22.535055 11.922166 20.715146
## [246] 25.644291 17.887771 23.332966 17.550824 30.692675 35.167886 32.118715
## [253] 28.642944 15.450453 13.094985 23.653344 25.573114  6.848566 23.622957
## [260] 26.543174 25.712725 25.252811 32.852013 28.369183 24.771812 28.342166
## [267] 18.654942 21.798055 18.059647 18.302895 23.288736 27.822587 13.533887
## [274] 26.271740 28.644508 19.120167 16.848601 15.656142 23.314955 21.373467
## [281] 26.278156 24.091449 25.773492 21.947641 17.627459 20.651570 27.704998
## [288] 22.785872 10.979430 30.102125 27.363808 23.766941 23.918741 25.538190
## [295] 21.377173 24.754604 29.253543 30.223561  7.363906 34.768262 26.002536
## [302] 26.335160 28.452467 21.998067 23.350247 22.037857 29.331420 16.680534
## [309] 16.209358 16.670845 15.082898 22.063640 20.807704 23.605948 24.723776
## [316] 26.654353 24.939442 24.307303 10.782240 19.127157 26.932868 23.158197
## [323] 18.023215 23.860016 18.991883 22.362972 27.444221 19.544762 21.975791
## [330] 25.359872 32.548087 20.740625 27.361631 16.095987 27.618862 16.448276
## [337] 20.246969 21.141860 15.852226 23.257301 23.645307 13.975173 17.449428
## [344] 22.332569 21.107949 21.856012 23.681698 20.700716 24.728543 20.230549
```

```r
w1 <- length(z1)
w1
```

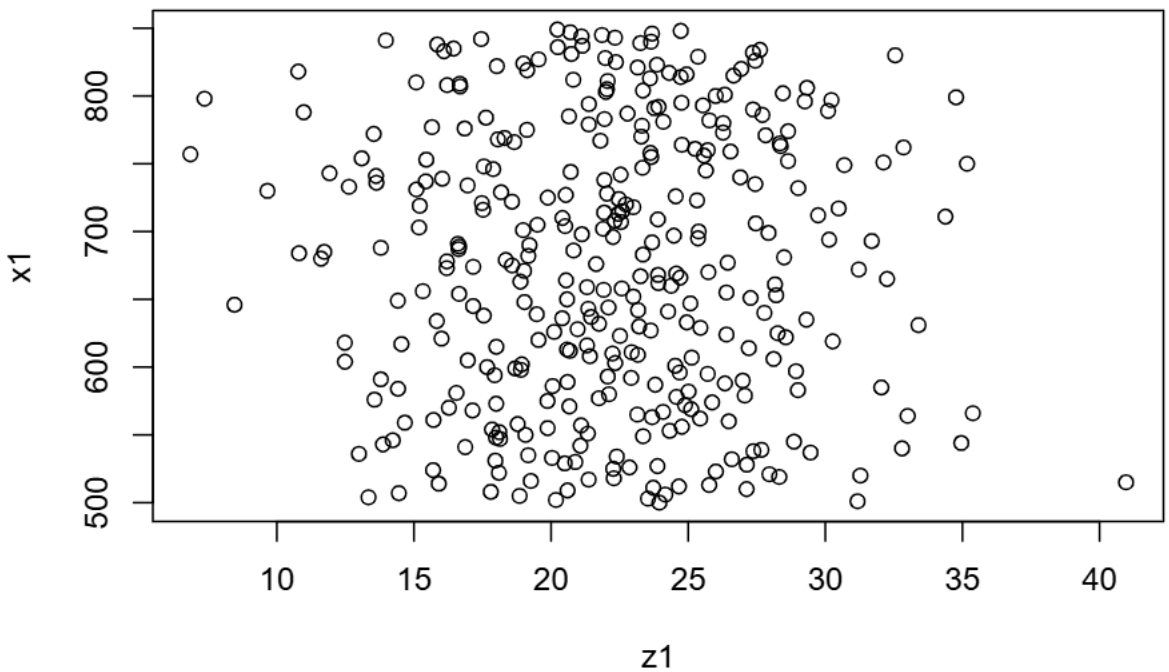
```
## [1] 350
```

```r
x1 <- 500:849
x1
```

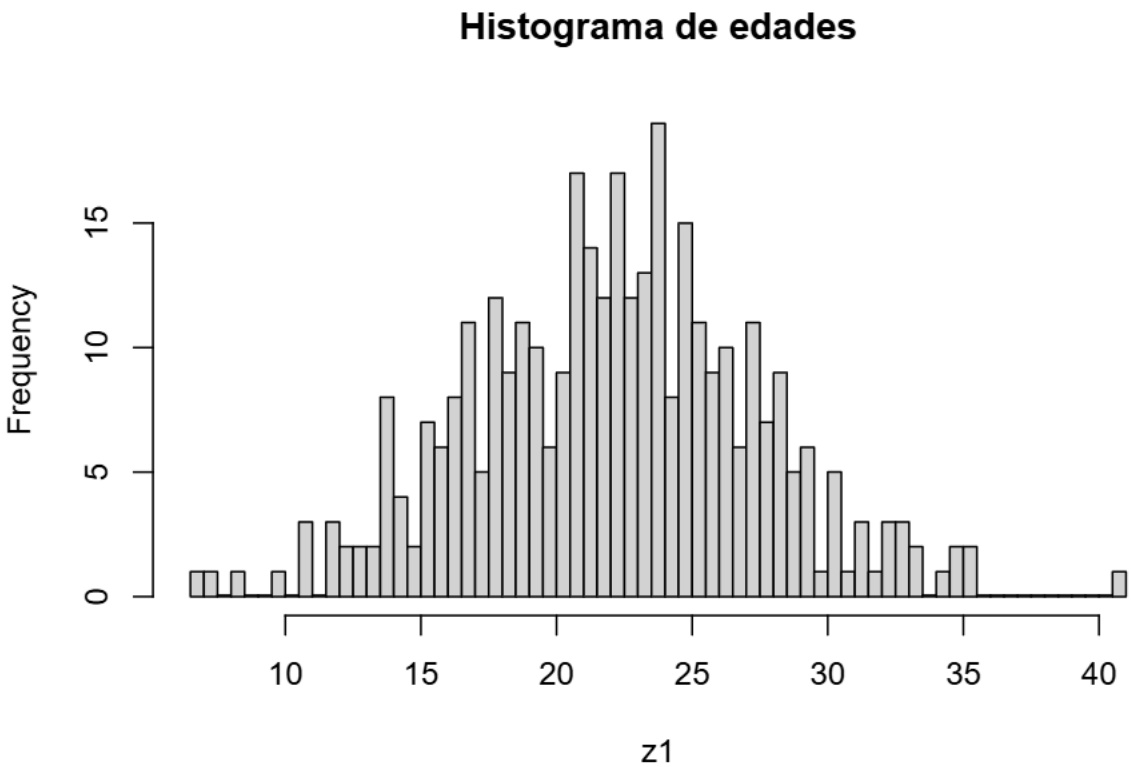
```
##   [1] 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517
##  [19] 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535
##  [37] 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553
##  [55] 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571
##  [73] 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589
##  [91] 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607
## [109] 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625
## [127] 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643
## [145] 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661
## [163] 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679
## [181] 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697
## [199] 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715
## [217] 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733
## [235] 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751
```

```
## [253] 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769
## [271] 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787
## [289] 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805
## [307] 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823
## [325] 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841
## [343] 842 843 844 845 846 847 848 849
```

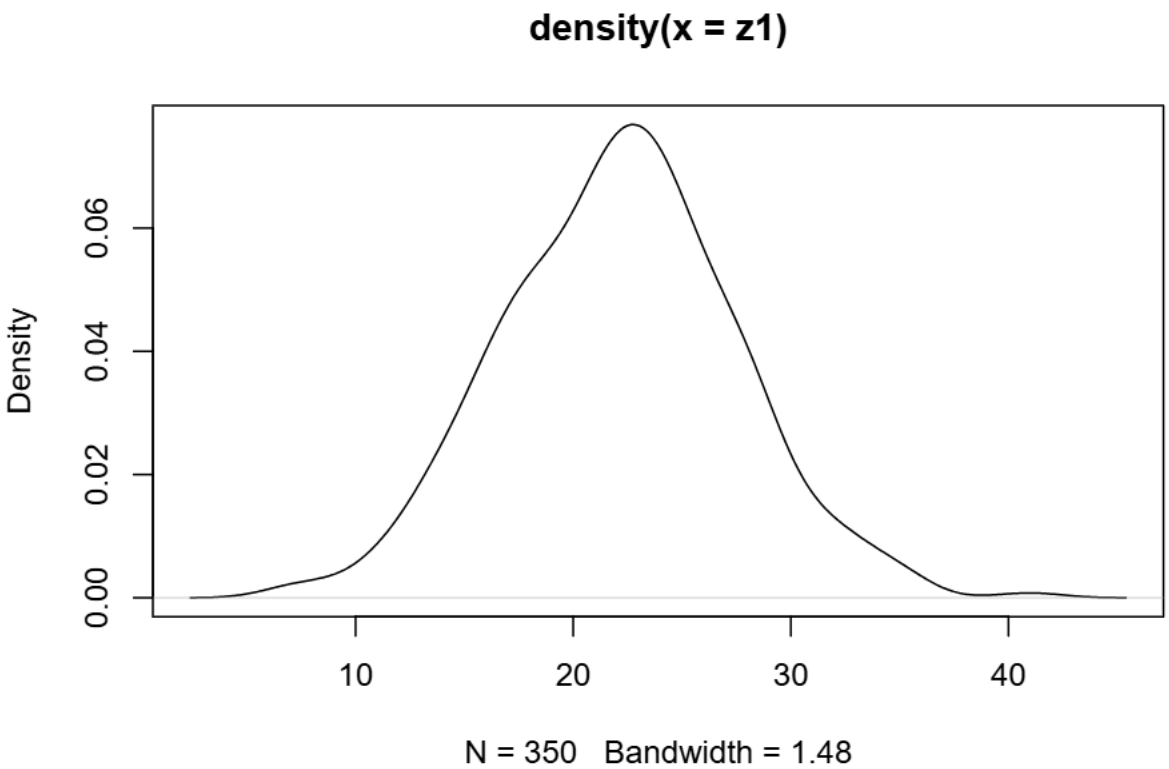
```r
plot(z1, x1)
```




```r
hist(z1, main = "Histograma de edades", breaks = 60)
```




```r
plot(density(z1))
```



Ejercicio 1 de algoritmos

Consigna:Las ultimas 3 cifras del DNI son 803, crear una variable que tenga ese numero

```r
DNI <- 803
```

Consigna: Crear un vector del 1 al 803.

```r
secuencia_dni <- 1:803
secuencia_dni
```

```
##   [1]   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18
##  [19]  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36
##  [37]  37  38  39  40  41  42  43  44  45  46  47  48  49  50  51  52  53  54
##  [55]  55  56  57  58  59  60  61  62  63  64  65  66  67  68  69  70  71  72
##  [73]  73  74  75  76  77  78  79  80  81  82  83  84  85  86  87  88  89  90
##  [91]  91  92  93  94  95  96  97  98  99 100 101 102 103 104 105 106 107 108
## [109] 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126
## [127] 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144
## [145] 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162
## [163] 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180
## [181] 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198
## [199] 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216
## [217] 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234
## [235] 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252
## [253] 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270
## [271] 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288
## [289] 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306
## [307] 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324
## [325] 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342
## [343] 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360
## [361] 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378
## [379] 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396
## [397] 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414
## [415] 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432
## [433] 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450
## [451] 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468
## [469] 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486
## [487] 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504
## [505] 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522
## [523] 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540
## [541] 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558
## [559] 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576
## [577] 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594
## [595] 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612
## [613] 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630
## [631] 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648
## [649] 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666
## [667] 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684
## [685] 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702
## [703] 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720
## [721] 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738
## [739] 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756
## [757] 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774
## [775] 775 776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792
## [793] 793 794 795 796 797 798 799 800 801 802 803
```

Ejercicio 2

Calcular la suma de todos los valores del vector secuencia_dni usando un `for`:

```
Total <- 0
valor_final <- length(secuencia_dni)

for (i in 1:valor_final) {
  Total <- Total + i
}

Total
```

```
## [1] 322806
```

Ejercicio 3

Repetir el ejercicio anterior pero en Phyton

Ejercicio 4 Secuencia

Generar secuencia de 2 en 2 hasta 100000

```
A <- numeric(50000)

for (i in 1:50000) {
  A[i] <- i * 2
}

head(A)
```

```
## [1]  2  4  6  8 10 12
```

```
tail(A)
```

```
## [1]  99990  99992  99994  99996  99998 100000
```

Secuencia con función de R:

```
B <- seq(2, 100000, by = 2)
```

Comparación de tiempos:

```
system.time({
  A <- numeric(50000)
  for (i in 1:50000) {
    A[i] <- i * 2
  }
})
```

```
##    user  system elapsed
##   0.005   0.000   0.004
```

```
system.time({
  B <- seq(2, 100000, by = 2)
})
```

```
##    user  system elapsed
##   0.001   0.000   0.001
```

Ejercicio 5 Fibonacci

```r
fib <- c(0, 1)

while (tail(fib, 1) < 1e6) {
  fib <- c(fib, fib[length(fib)] + fib[length(fib)-1])
}

fib
```

```
##  [1]       0       1       1       2       3       5       8      13      21
## [10]      34      55      89     144     233     377     610     987    1597
## [19]    2584    4181    6765   10946   17711   28657   46368   75025  121393
## [28]  196418  317811  514229  832040 1346269
```

```r
length(fib)
```

```
## [1] 32
```

Cantidad de iteraciones necesarias:

```r
length(fib)
```

```
## [1] 32
```

Ejercicio 7 Burbuja vs sort

```r
x <- sample(1:20000, 20000)

burbuja <- function(x) {
  n <- length(x)
  for (j in 1:(n-1)) {
    for (i in 1:(n-j)) {
      if (x[i] > x[i+1]) {
        temp <- x[i]
        x[i] <- x[i+1]
        x[i+1] <- temp
      }
    }
  }
  return(x)
}
```

Comparación:

```r
system.time({
  res1 <- burbuja(x)
})
```

```
##    user  system elapsed
##  20.709   0.054  20.878
```

```r
system.time({
  res2 <- sort(x)
})
```

```
##    user  system elapsed
##       0       0       0
```

Ejercicio 8 Penitencia de Newton

Método 1 (for):

```r
n <- 1e6

system.time({
  suma1 <- 0
  for (i in 1:n) {
    suma1 <- suma1 + i
  }
})
```

```
##    user  system elapsed
##   0.017   0.000   0.017
```

Método 2 (fórmula matemática):

```r
system.time({
  suma2 <- n * (n + 1) / 2
})
```

```
##    user  system elapsed
##       0       0       0
```

Comparación:

```r
suma1
```

```
## [1] 500000500000
```

```r
suma2
```

```
## [1] 500000500000
```

Conclusión

El método matemático es muchísimo más eficiente que el uso de bucles.